

Which LLM are we allowed to use?

Emil Lindfors | August 11, 2026

A few months ago someone at work asked which LLM our consultants should be allowed to use, and on which client projects. Nobody could answer it cleanly. I assumed for a while that this was a procurement problem and that somebody just needed to sit down with a spreadsheet.

It wasn't a procurement problem. There were two separate decisions tangled up in the question, and every discussion we had went in circles until we pulled them apart.

Two different decisions

Internal consultant tooling is how our own people use models on their own machines. We are the decision-maker. The cost is noise against billable hours. What binds us is governance: can we say, in a sentence, what our consultants are allowed to send where, and can we show it to someone who asks?

Client deliveries are what we recommend, build and operate for someone else. Here the decision-maker is the client's legal department, the cost is part of a commercial model, and what binds us is an approved endpoint plus an audit trail. The regulatory frame is identical to the internal case, but the threat model, the cost profile and the person who has to say yes are all different, which is why answering both at once never worked.

Where the data goes

The observation that reorganised our thinking came out of three unrelated investigations that all landed in the same place: a Power BI integration, a data-warehouse agent, and some sandboxing work. In all three, what determined where client data ended up was the LLM endpoint the tool was pointed at, and the choice of tool barely mattered.

A local integration running on your laptop doesn't bypass row-level security or role-based access control, which is good, but that isn't the boundary that matters. Whatever the tool returns goes into the conversation context, and the context is sent onward to whichever provider is configured. Microsoft says this explicitly in their own documentation for the Power BI MCP server. **"It runs locally" is necessary and nowhere near sufficient.**

Once you accept that, **the highest-value control in the whole stack is getting the client to approve a named endpoint.** Most of the other questions fall into place behind

that one, and which tool or IDE or agent framework you use becomes a preference. Which was fine as a conclusion until we looked at what that implied about the tools we were already using.

The bundle problem

Commercial coding agents ship as a bundle: one vendor's client, locked to one vendor's endpoint. That's a reasonable product decision and a bad fit for a consultancy, because the endpoint is exactly the variable our clients need to control.

We do a lot of work inside a client's own environment: their Microsoft 365 tenant, their cloud account, their data platform. When we do, the endpoint choice isn't ours at all. It changes the commercial model and it changes our role under any AI management standard. A tool that can only talk to one vendor's API can't follow us there.

So we forked one.

We took OpenAI's Codex CLI, which is open source under Apache-2.0 and a good agent, and built an internal fork. The fork is deliberately small: a rebrand, telemetry off by default, its own config directory, and a handful of patches that restore the ability to point the agent at any OpenAI-compatible endpoint. Local models on our own GPU server. Azure OpenAI with corporate single sign-on. OpenRouter. A European sovereign inference provider. A model running inside a client's own tenant.

Upstream had recently gone all-in on OpenAI's newer Responses API and dropped support for the older Chat Completions protocol. That one removal is what locks the agent to a single vendor, because almost every other inference service in the world speaks Chat Completions: aggregators, GPU clouds, local runtimes. Restoring it was about **a thousand lines of code**. I've written up [how that works, and how to do it yourself](#), in a technical companion to this post.

What changed afterwards was that "which model provider?" stopped being an architectural question and became a config line. We can now answer a client's legal department with "whichever endpoint you approve" and mean it.

What it cost

This is where it's easiest to build an argument that doesn't survive contact with a room full of people who know the numbers, so I want to be careful about it.

Self-hosting almost never wins on price. Compared to cheap open-weight APIs at roughly **\$0.14–0.50 per million tokens**, the break-even for a single high-end GPU sits in the **billions of tokens per month**. Western frontier APIs put the crossing point somewhere more reachable, but only if you assume **60–70% sustained utilisation**. A node running eight hours a day pays **five times as much per useful token**, and that's where

most self-hosting business cases die. If you present self-hosting as a cost argument, the utilisation assumption belongs on the same slide, or it gets taken apart in front of you. The argument that holds up is availability and control.

At consultancy scale, tooling cost isn't the interesting variable anyway. A model subscription runs a few hundred kroner per consultant per month, low thousands with heavy agentic usage on consumption pricing. A fully loaded senior consultant costs on the order of **1.4 MNOK a year**. The tooling is something like **0.3%** of that, which is noise, and optimising it is optimising the wrong thing.

There's one cost variable worth watching and it isn't price per token. Model efficiency now moves cost-per-task more than pricing does. On one recent agentic coding benchmark the most expensive model per token was the *cheapest per completed task*, because it used **a quarter of the tokens and half the steps**. Related: reasoning effort is a cost allocation rather than a quality switch. Run everything at maximum and you burn the budget on tasks a low setting solves just as well.

None of that is worth asserting without measuring it, so I built the telemetry to do so: two days of real use came to **\$0.184 across 5.97 M tokens**, a third of which were prompt-cache hits. **How that stack is put together, and the three ways I got the numbers wrong first**, is a separate post.

We didn't start any of this to save money. The savings turned up anyway, which is a pleasant result and still a bad thing to lead a proposal with.

Why cheap open-weight models changed things

Some APIs are simply not available to us, for reasons that have nothing to do with quality or price.

China has no adequacy decision under GDPR. DeepSeek publishes no standard contractual clauses. Italy's data protection authority blocked the service **within 72 hours** of review, and investigations opened in **more than a dozen** European jurisdictions. Chinese organisations are obliged under national intelligence law to assist state intelligence work, and no contract clause overrides that. For a consultancy handling client data, `api.deepseek.com` is a door that's closed rather than a risk to be weighed.

But **the model and the API are two different things**. Open weights running on a European endpoint, under a European processor agreement, is a completely different compliance scenario from the same weights behind a Chinese API. Same matrices, *entirely different data flow, entirely different legal analysis*.

That's why the recent generation of cheap, strong open-weight models matters commercially and not just technically. A model like DeepSeek V4 Flash costs on the order of **\$0.09 per million input tokens and \$0.18 per million output**, roughly *an order of magnitude* below frontier pricing, and it's good enough to drive an agentic coding loop all day. Serve

those weights from an endpoint whose jurisdiction and retention terms your client's lawyers have approved, and you have something that wasn't available eighteen months ago: a stack that is cheap and defensible at the same time.

We're not out on a limb here either. By one industry estimate, **over 40% of large European enterprises** plan to deploy at least one open-weight model on private infrastructure by the **end of 2026**.

What flexibility doesn't get you

Being able to point at any endpoint is necessary and it is not compliance. Four things I'd want any stakeholder reading this to take away.

Localisation is not compliance. Forcing processing into the EU solves the localisation question and none of the others. Lawful basis, the processor agreement, data subject rights and your own documentation obligations sit with the controller regardless of where the bytes are processed. This is the point vendor marketing skips most often.

The set of approved models has to be pinned, with a named owner. If you rely on an aggregator's routing controls, note that the set of EU-eligible models is dynamic. Base your posture on "whatever qualifies today" and your compliance position changes without you doing anything. The mitigation is cheap and concrete: pin an explicit allow-list, treat a model change as a controlled change, and name the person who owns the list and the cadence at which it's reviewed. Without an owner and a cadence, "pinned models" is an intention rather than a control. That applies just as much to internal tooling as to client deliveries.

Metadata is still personal data. Even with region routing, zero-data-retention and training opt-out all enabled, aggregators retain operational metadata for billing: timestamps, model, token counts, latency. If that can be linked to a data subject, it's in scope.

Pseudonymisation is the one control that survives every vendor decision. Minimise what the prompt doesn't need, tokenise direct identifiers, keep the re-identification key under your own control, and treat the model call as an operation on pseudonymised text. It shrinks the blast radius so that a downstream retention failure is far less serious, and it's the kind of technical measure an auditor expects to see. It's also portable, in that it holds no matter which endpoint you end up on. The caveat, which we should say out loud before a regulator does: the legal bar for true anonymisation is high, and pseudonymised data is still *personal data*, so this shrinks the risk without removing a single obligation.

Things I'd be careful claiming

The EU AI Act **does not require EU-only compute** for most high-risk systems. Compute in the EU makes audit-log obligations easier to satisfy, which is a real advantage, but

it's not a legal requirement. Someone in any serious client meeting will know this, and overstating it costs you credibility on everything else you say.

Similarly, an AI management system certification such as ISO/IEC 42001 certifies the management system, not particular models. Changing model or hosting doesn't invalidate a certificate; it triggers change management, a refreshed risk assessment and supplier due diligence. That's a process cost of perhaps **a day**. Worth knowing in both directions, since the standard doesn't restrict your options so much as restrict how you make and document the choice.

On timing: as of writing, the AI Act's high-risk requirements have slipped to **late 2027**, with Norwegian implementation expected around **mid-2027**. The prohibitions and the AI-literacy requirement are unaffected by the delay. Practically, that means compliance-as-a-sales-driver is a weaker argument over the next twelve months than it looked a year ago, and anyone still quoting a 2026 date should update the message rather than keep using a date that's now wrong.

Where we landed

None of this was a grand plan. It started as an experiment, mostly curiosity about whether we could run a coding agent against models we controlled, and the useful findings came out sideways. We set out to look at security and cost, and the thing that ended up mattering most was being able to change endpoint at all.

The stack we run now: a small, documented fork of an open-source agent; local models on our own hardware for anything that shouldn't leave the building; a cheap open-weight model on a European endpoint for the bulk of day-to-day work; the enterprise API of a frontier vendor when the task warrants it; and whatever the client has approved when we're working inside their environment. One binary, one config file, one pinned model list.

I'm still not sure where the line sits for client work that involves personal data at any real volume, and I'd like to hear from anyone who has drawn it somewhere defensible. The technical companion, covering how the fork works and how to set the same thing up in about ten minutes, is [here](#).